

AI Chat Proxy — Architecture Analysis

Project: AI Chat Proxy

Technology: JavaScript, Node.js, Express, OpenAI API

Location: /home/dominic/Documents/ai-chat-proxy

Report #: 2/20

Architecture Type

Thin proxy middleware server

Layers

Client → Express Proxy → OpenAI API

Design Patterns

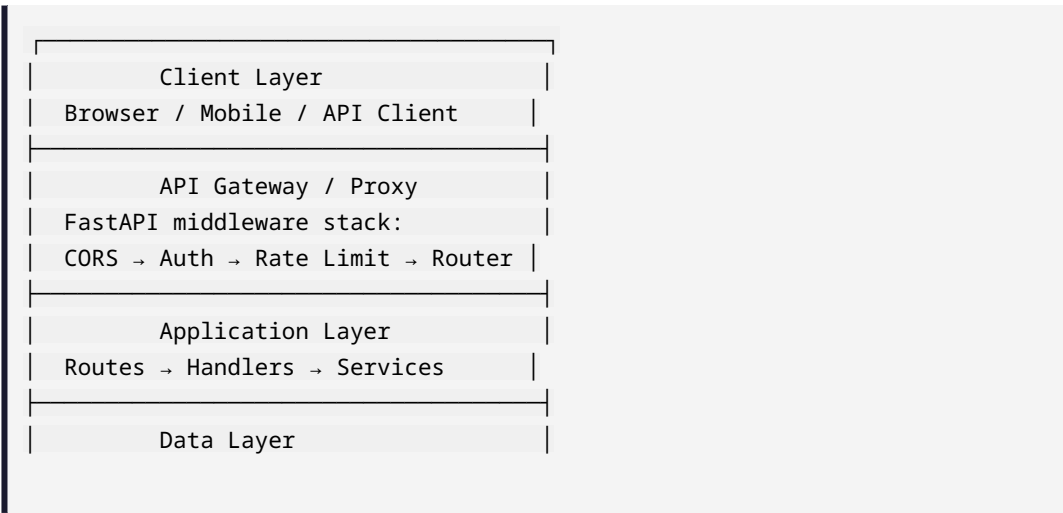
Reverse proxy pattern

Scalability

Depends on upstream OpenAI rate limits

System Architecture

Layer Architecture



| SQLAlchemy ORM → SQLite/PostgreSQL |

Component Interaction

AI Chat Proxy Component Architecture

| API Layer (FastAPI)

| Service Layer (Business Logic)

| Data Layer (SQLAlchemy + SQLite)

| Client Layer (None)

Request Lifecycle

1. **HTTP Request** arrives at FastAPI
2. **CORS middleware** checks origin headers
3. **Auth dependency** extracts and validates JWT
4. **Rate limiter** checks request count per IP
5. **Router** matches path to handler
6. **Handler** executes business logic
7. **Database session** commits or rolls back
8. **Response** returned to client

Dependency Graph

AI Chat Proxy Dependency Flow:

Client → FastAPI → Router → Handler → SQLite → Response

Design Patterns Used

Pattern	Location	Purpose
Dependency Injection	FastAPI Depends()	DB sessions, auth
Repository Pattern	SQLAlchemy models	Data abstraction
Middleware Chain	FastAPI middleware	Cross-cutting concerns
Router Module	APIRouter include	Separation of concerns